Analysis Service Working

1. What We Send to Gemini API:

We send a structured summary, not raw code. Here's the exact flow:

1. Data Collection (Your Code Already Does This)

Project Analysis Results:

- Project Type: JAVA
- Main Language: Java
- Build Tool: Maven
- Architecture: FULL_STACK
- Key Files: [pom.xml, src/main/, README.md]
- Dependencies: [Java project - check pom.xml]
- File Structure: {Map of file paths → "FILE" or "DIR"}
- Sub-projects: [backend (JAVA), frontend (NODE_JS)]

2. What Gets Sent to Gemini:

Project Details:

- Name: MySpringApp
- Type: JAVA (Full-stack)
- Main Files: pom.xml, src/main/java/, src/components/
- Key Directories: controllers/, services/, models/, public/
- Build: Maven
- Has: Backend (Java/Spring), Frontend (React components)
- Special Files: Dockerfile, application.properties

3. How We Read Project Files:

```
private Map<String, String> scanProjectStructure(String localPath) throws IOException {
    // This ONLY collects FILE NAMES and PATHS, not file content!
```

```
    // Example output:
    // {
    //   "src/main/java/com/example/Main.java": "FILE",
    //   "src/components/Header.js": "FILE",
    //   "pom.xml": "FILE",
    //   "src/": "DIR"
    // }
    // No file content read here!
}
```

Why This Works:

1. **Token Limit Safe**: We send ~100-500 words, not thousands of lines of code
2. **AI Doesn't Need Code**: Gemini only needs to understand the **project structure** and **technology stack** to generate a good README
3. **Smart Filtering**: We only mention key/important files, not every single file

Complete Example - What Actually Goes to Gemini:

The prompt sent to Gemini looks like:

```
String prompt = """
Project Details:
Project Name: E-commerce Backend
Repository: github.com/example/ecom
Template Domain: WEB_SOFTWARE
Project Type: JAVA
Architecture: BACKEND_ONLY
Main Language: Java
Build Tool: Maven
Key Files: [pom.xml, src/main/java/, application.properties, Dockerfile]
Sub-projects: []
```

Dependencies: [Spring Boot, Hibernate, PostgreSQL]

File Structure: Contains controllers/, services/, models/, repository/ directories

""";

Key Insight:

- **File scanning** (Files.walk) → Gets **file paths only** (not content)
- **Analysis** → Determines project type/structure from **file presence**
- **Prompt building** → Creates a **textual summary** of findings
- **Gemini receives** → Only the summary, understands the "shape" of the project
- **Output** → Context-aware README based on project type/structure

What We NEVER Send:

- Actual source code content
- Entire file contents
- Binary files or libraries
- All files in node_modules/

The scanProjectStructure() returns:

```
{
    "src": "DIR",
    "src/main": "DIR",
    "src/main/java": "DIR",
    "src/main/java/com": "DIR",
    "src/main/java/com/example": "DIR",
    "src/main/java/com/example/dto": "DIR",
    "src/main/java/com/example/dto/UserDTO.java": "FILE",
    "src/main/java/com/example/dto/ProductDTO.java": "FILE",
    "src/main/java/com/example/dto/OrderDTO.java": "FILE",
    "src/main/java/com/example/controller": "DIR",
    "src/main/java/com/example/controller/UserController.java": "FILE",
    "src/main/java/com/example/controller/ProductController.java": "FILE"
}
```

Even Smarter: identifyKeyFiles()

Another function filters this further:

```
    private List<String> identifyKeyFiles(String localPath) {
        (Only checks for IMPORTANT files, not all files)
        String[] importantFiles = {
            "README.md", "package.json", "pom.xml", "build.gradle",
            "requirements.txt", "Dockerfile", "docker-compose.yml",
            ".env", "src/", "main.py", "app.js", "index.js", "Main.java",
            "application.properties", "config/", "public/", "src/main/", "src/test/"
        };
        (Returns: ["pom.xml", "src/main/", "application.properties"])
    }
```

Final Gemini Prompt Looks Like:

Project has:

- dto/ directory with multiple DTO classes
- controller/ directory with REST controllers
- src/main/java/ structure
- pom.xml file
- application.properties

This indicates a Java Spring Boot application with:

- Data Transfer Objects for API requests/responses
- REST controllers for HTTP endpoints
- Maven build system
- Configuration via properties files

Why This Works So Well

1. AI Training Data

Gemini has been trained on **millions** of GitHub projects. When it sees:

- pom.xml + src/main/java/ + controller/ + dto/
  → It **immediately knows** this is a Spring Boot REST API

2. Common Patterns Recognition

For a Java project, the README **doesn't need** to know your specific business logic. It needs:

- **Setup instructions** (mvn install, mvn spring-boot:run)
- **Project structure** explanation
- **API documentation** format
- **Database configuration**
- **Testing commands**

3. Real-World Example:

Your project structure tells Gemini:

Input: Java + Maven + controllers/ + dto/ + application.properties

↓

AI's Understanding: "Spring Boot REST API with DTOs and controllers"

↓

Output README includes:

• Prerequisites: Java 17, Maven, PostgreSQL

• Running: `mvn spring-boot:run`

• API endpoints section

• DTO structure explanation

• Database setup

4. What Gemini INFERS (Doesn't Need Code):

From dto/UserDTO.java **filename alone**, Gemini knows:

- This is a Data Transfer Object

- It's probably used in API requests/responses
- Should be mentioned in the "API Structure" section
- Follows Java naming conventions

The Key Insight:

READMEs are 80% boilerplate + 20% project-specific. The AI needs to know:

1. **Technology stack** (Java/Spring vs Node/Express vs Python/Django)
2. **Project type** (API vs Web App vs Library)
3. **Architecture** (MVC, Microservices, etc.)
4. **Key files** that matter for setup

Your file/folder names give it **exactly that**!